Verilog / SystemVerilog

recto · le langage

`mot-clé type $système // commentaire «à remplir»` SV

Opérateurs

1 · opérateurs · types

Arithm.	`+ - * / %`	`**` puissance
Relation.	`> < >= <=`	résultat sur 1 bit
Égalité	`== !=`	x ou z ⇒ résultat x
	`=== !==`	compare aussi x et z
	`==? !=?` SV	x / z à droite = joker
Logiques	`&& || !`	résultat sur 1 bit
Bit à bit	`& | ^ ~ ~^`	et, ou, xor, non, xnor
Réduction	`&a |a ^a`	tous les bits → 1 bit
	`~&a ~|a ~^a`	`^a` = parité
Décalages	`<< >>`	logiques (insère des 0)
	`<<< >>>`	arithm. (`>>>` garde le signe)
Condition	`c ? a : b`	a si c vrai, sinon b
Ensemble	`a inside {1,4}` SV	appartient à l'ensemble
Incrément	`i++ i-- +=` SV	comme en C
Concat.	`{a, b[2:0]}`	assemble les bits
Réplication	`{4{2'b01}}`	= `8'b01010101`
Affectation	`= <=`	bloquante · non bloquante
Sélection	`v[7:4]`	intervalle de bits
	`v[i+:4]`	4 bits à partir de i

Littéraux

Format	`8'hFF`	`<taille>'<base><valeur>`
Bases	`'b 'o 'd 'h`	binaire, octal, décimal, hexa
Signé	`4'sb1010`	`s` = signé (vaut −6)
Défaut	`1973 -123`	décimal, 32 bits, signé
Sans taille	`'hcafe`	32 bits
Séparateur	`16'hBE_EF`	`_` ignoré (lisibilité)
x / z	`4'b10xz`	inconnu / haute impédance
Remplissage	`'0 '1 'x 'z` SV	tous les bits à la valeur

Types Verilog

Logique 4 valeurs : `0` `1` `x` inconnu · `z` haute impédance

Nets	`wire tri`	fil · bus trois états
	`wand wor`	ET / OU câblé
	`tri0 tri1`	tiré à 0 / à 1
	`supply0 supply1`	masse / alimentation
Variables	`reg`	affectée dans un process
	`integer`	entier 32 bits signé
	`real`	flottant
	`time realtime`	temps de simulation

Vecteurs et mémoires

```
wire [31:0] a;                  // vecteur 32 bits
reg  [15:0] b;                  // b[15] = MSB
integer t [0:100];              // 101 entiers
reg  [31:0] m [0:1023];         // 1024 mots de 32 bits
reg  [31:0] m2 [0:1023][1:20];  // tableau 2D
```

Conversions (cast) SV

```
int'(3.7)       // = 4 (réel -> entier, arrondi)
8'(x)           // taille : tronque / étend
signed'(x)      // relit comme signé     ($signed)
unsigned'(x)    // relit comme non signé ($unsigned)
```

Types SystemVerilog

2 · types SystemVerilog

type	états	bits	signe	
`bit`	2	variable	non signé	vecteur 2 états
`logic`	4	variable	non signé	remplace `reg` / `wire`
`byte`	2	8	signé	ou caractère ASCII
`shortint`	2	16	signé	
`int`	2	32	signé	
`longint`	2	64	signé	
`integer`	4	32	signé	
`time`	4	64	non signé	

Chaînes : string

```
s.len()          s.getc(i)        s.putc(i, c)
s.substr(i, j)   s.compare(s2)    s.toupper()
s.itoa(i)        s.hextoa(i)      s.atoi()
```

Énumérations et structures

```
typedef enum {RED, YELLOW, GREEN} color_t;
typedef enum logic [1:0] {IDLE, RUN} state_t;  // codé
color_t c = RED;
c = c.next();  c.name();          // YELLOW, "YELLOW"
typedef struct packed {           // packed : un vecteur
  bit [7:0]  opcode;
  bit [23:0] addr;
} instr_t;
instr_t ir;
ir.opcode = 8'hAE;  ir = 32'hAE000010;   // champ · tout
typedef union packed { int i; bit [31:0] b; } mot_t;
```

Tableaux

```
bit   [7:0] c1;               // packed : vecteur de bits
real  u [7:0];                // unpacked : tableau
int   t [8][32];              // = [0:7][0:31], notation C
logic [7:0] mem [0:255];      // mémoire 256 × 8 bits
data = mem[addr];             // lit le mot n°addr
A = B;  A[i:j] = B[i:j];  A == B;   // copie, tranche, ==
```

Dynamiques

```
bit [3:0] nib[];              // taille fixée au run
nib = new[12];                // alloue 12 éléments
nib = new[13](nib);           // agrandit, garde les 12
nib.size();  nib.delete();    // taille · vide tout
```

Files (queues)

```
int q[$] = {3, 2, 7};         // file initialisée
bit b[$:255];                 // 256 éléments au plus
q.push_back(1);  q.push_front(0);
x = q.pop_front();  y = q.pop_back();
q.insert(i, v);  q.delete(i);  q.size();
x = q[0];  y = q[$];          // premier · dernier
```

Tableaux associatifs

```
bit [20:0] tab[string];       // indexé par chaîne
tab["toto"] = 21'h13cafe;
if (tab.exists("titi")) tab.delete("titi");
foreach (tab[k]) $display("%s", k);  // clés triées
```

Aussi : `.num()` `.first(k)` `.next(k)` `.last(k)` `.prev(k)` ; `int t[*]` : index quelconque.

Structures de contrôle

3 · contrôle · temps

```
if (cond) begin
  instructions
end else if (cond2) begin
  instructions
end else begin
  instructions
end

case (expression)
  v1      : instr;
  v2      : begin instrs end
  v3, v4  : instr;       // même branche
  default : instr;       // évite les latchs
endcase

for (int i = 0; i < N; i++) begin ... end
repeat (n)      begin ... end
while (cond)    begin ... end
foreach (tab[i]) begin ... end
do begin ... end while (cond);
forever begin ... end          // boucle infinie
```

`casez` : z ou ? = joker · `casex` : x et z jokers.

Délais et évènements

```
`timescale 1ns/1ps        // unité / précision
#20;                      // attendre 20 unités
#10 a = 1;                // retarde l'instruction
wait (cond) #(d) instr;   // attend cond vraie
@(évènement) instr;       // attend l'évènement
@(posedge clk);           // front montant
@(negedge rst_n);         // front descendant
@(a or b)  @(a, b)  @*    // liste de sensibilité
event done;               // évènement nommé
-> done;                  // le déclenche
wait (done.triggered);    // l'attend (sans course)
```

Tâches et fonctions

```
task do_something;            // style Verilog
  input  REQ;
  output GNT;
  begin instructions end      // peut consommer du temps
endtask

task do_something(input  int          DATA,   // style SV
                  output logic [12:0] RESULT);
  instructions
endtask

function [15:0] max_value;    // style Verilog
  input [15:0] a, b;
  begin max_value = a < b ? b : a; end
endfunction

function bit [15:0] max_value(bit [15:0] a, b);
  return a < b ? b : a;       // style SV
endfunction
```

task : peut contenir `#`, `@`, `wait` ; pas de valeur de retour. **function** : s'exécute en temps nul, renvoie une valeur.

Verilog / SystemVerilog

verso · conception et simulation (→ vérif : fiche UVM)

mot-clé type $système // commentaire «à remplir» SV

Structure d'un module

4 · module

```
module my_module_name (...);
  parameter nom = valeur_défaut;

  input  signal;
  output signal;

  assign nom_wire = expression;

  initial begin instrs end  // 0, 1 ou plusieurs
  always  begin instrs end  // 0, 1 ou plusieurs
  task, function

  generate
    genvar i;
    for (i = 0; i < 10; i++) begin : my_name
      always, initial, instances, signaux
    end
  endgenerate
endmodule
```

Instanciation

```
mod #(.N(8)) u1 (.clk(clk), .q(q)); // par nom (mieux)
mod u2 (clk, q);                    // par position
```

Bloquant vs non bloquant

`=` **bloquant** : effectué avant l'instruction suivante
`<=` **non bloquant** : mis à jour à la fin du pas de temps (delta suivant)

```
// Non bloquant : registre (séquentiel)
always @(posedge clk) begin
  a <= b;
  b <= a;                 // échange a et b
end

// Bloquant en séquentiel : à éviter
always @(posedge clk) begin
  tmp = b;  b = a;  a = tmp;
end
```

⚠ **Piège :** séquentiel ⇒ <=, combinatoire ⇒ =. Jamais les deux dans un même bloc.

Mémoire (RAM, ROM)

```
logic [7:0] mem [0:255];
initial $readmemh("init.hex", mem);   // contenu initial
always_ff @(posedge clk) begin
  if (we) mem[addr] <= din;           // écriture
  dout <= mem[addr];                  // lecture synchrone
end
```

`init.hex` : un mot hexa par ligne (`$readmemb` : binaire). Lecture synchrone et écriture sur un front : reconnu comme bloc RAM à la synthèse.

Règles de synthèse

`always_ff`	séquentiel : <=, une horloge (+ reset asynchrone dans la liste)
`always_comb`	combinatoire : =, sortie affectée dans toutes les branches
`# initial`	ignorés (simulation seule ; `initial` admis sur FPGA)
Un pilote	un signal n'est affecté que dans un seul `always` ou `assign`

Circuit combinatoire

5 · circuits

```
module mux2 (input  logic a, b, sel,
             output logic f);
  always_comb
    if (sel == 0) f = a;
    else          f = b;
endmodule
```

Équivalent : `assign f = sel ? b : a;`

⚠ **Piège :** affecter la sortie dans *toutes* les branches, sinon latch.

Élément séquentiel

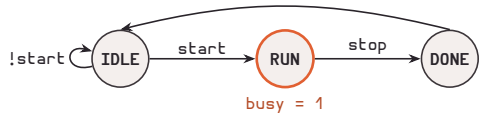
```
module flipflop_syncr (d, q, clk, rst_n);
  input d, clk, rst_n;
  output logic q;
  always_ff @(posedge clk)
    if (rst_n == 0) q <= 0;  // reset synchrone
    else            q <= d;
endmodule
```

Reset asynchrone : `always_ff @(posedge clk or negedge rst_n)`.

Compteur avec enable

```
always_ff @(posedge clk)
  if (!rst_n)  cnt <= '0;       // '0 : tous les bits à 0
  else if (en) cnt <= cnt + 1;
```

Machine à états (FSM)




```
module fsm (input  logic clk, rst_n, start, stop,
            output logic busy);
  typedef enum logic [1:0] {IDLE, RUN, DONE} state_t;
  state_t state, next;

  // 1. registre d'état (séquentiel)
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) state <= IDLE;
    else        state <= next;
  // 2. état suivant (combinatoire)
  always_comb begin
    next = state;                  // défaut : on reste
    case (state)
      IDLE    : if (start) next = RUN;
      RUN     : if (stop)  next = DONE;
      DONE    :            next = IDLE;
      default :            next = IDLE;
    endcase
  end
  // 3. sorties (Moore : fonction de l'état seul)
  assign busy = (state == RUN);
endmodule
```

Mealy : sorties fonction de l'état *et* des entrées → les calculer dans le `always_comb`. `next = state` par défaut évite les latchs.

Testbench

6 · simulation

```
`timescale 1ns/100ps
module test_adder;
  logic [3:0] a, b;                  // signaux internes
  logic [4:0] s;
  adder dut (.a(a), .b(b), .s(s));   // instanciation

  logic clk = 1'b1;
  always #5 clk = ~clk;              // période 10 ns

  initial begin
    a = 0;  b = 3;                   // à 0 ns
    #10 a = 1;                       // à 10 ns
    @(posedge clk) a = 2;            // au front suivant
    $display("t=%0t s=%h", $time, s);
    $finish;                         // fin de simulation
  end
endmodule
```

`$monitor` affiche à chaque changement · `$stop` pause · `$urandom` aléatoire.

Formats de $display

`%b %o %d %h`	binaire, octal, décimal, hexa
`%s %t %0d`	chaîne, temps, sans espaces

Interfaces

```
interface apb_if (input logic clk);
  logic        psel, penable, pready;
  logic [31:0] paddr;
  modport slave  (input  psel, penable, paddr,
                  output pready);   // master : inverse
  clocking cb @(posedge clk);      // vue synchrone
    default input #1step output #1;
    output psel, paddr;            // front + #1
    input  pready;                 // juste avant
  endclocking
endinterface
module dut (apb_if.slave s); ...s.psel... endmodule
apb_if bus (clk);                  // instance (top)
dut u (.s(bus));
virtual apb_if vif = bus;          // handle (classes)
@(vif.cb);  vif.cb.psel <= 1;      // synchrone
```

Processus parallèles

```
fork
  instruction;            // branches lancées
  begin a; b; end         // en parallèle
join                      // attend toutes
// join_any : la 1re · join_none : aucune
wait fork;                // attend les fils restants
disable fork;             // tue les fils restants
```

Communication entre processus

```
mailbox #(string) mb = new();  // FIFO, new(N) : bornée
mb.put("hi");  mb.get(s);      // bloquants
mb.try_get(s); mb.num();       // non bloquant · taille
semaphore sem = new(1);        // 1 clé
sem.get(1); section critique sem.put(1);
```